

SQL & Databases

A Clean Sheet for Beginners

1 · What is a Database?

A **database** is an organised collection of data stored and accessed electronically.
Think of it like a super-powered spreadsheet that can store **millions of rows**, link related data together, and let multiple users read/write at the same time.

Why use a database instead of a spreadsheet?

Spreadsheet	Database
Manual, error-prone links	Automatic relationships & integrity
Slows down with large data	Handles millions of rows easily
One user at a time (usually)	Many users simultaneously
No access control	Fine-grained user permissions

2 · Tables, Rows & Columns

A database holds **tables** — like sheets in a workbook.
Columns (fields) define what kind of data is stored — e.g. name, age, email.
Rows (records) are individual entries — e.g. one customer per row.
Every table should have a **Primary Key** — a unique ID for each row.

Example — customers table

customer_id (PK)	first_name	last_name	email	city
1	Alice	Mokoena	alice@email.com	Johannesburg
2	Bob	Dlamini	bob@email.com	Cape Town
3	Carol	Nkosi	carol@email.com	Durban

3 · Relationships

One-to-One (1:1)	One-to-Many (1:N)	Many-to-Many (M:N)
Each row in Table A matches exactly one row in Table B. <i>Example: Person ↔ Passport</i>	One row in A links to many rows in B. <i>Example: Customer → Orders</i>	Many rows in A link to many rows in B (via a junction table). <i>Example: Students ↔ Courses</i>

4 · Indexes & Constraints

Constraint / Index	What it does	Example
--------------------	--------------	---------

PRIMARY KEY	Uniquely identifies each row; auto-indexed	id INT PRIMARY KEY
NOT NULL	Column must always have a value	name VARCHAR(50) NOT NULL
UNIQUE	All values in the column must be different	email VARCHAR(100) UNIQUE
CHECK	Value must pass a condition	age INT CHECK (age >= 0)
FOREIGN KEY	Links a column to another table's PK	FOREIGN KEY (cust_id) REFERENCES customers(id)
INDEX	Speeds up searches on a column	CREATE INDEX idx_name ON users(last_name)
DEFAULT	Fills in a value if none is provided	status VARCHAR(10) DEFAULT 'active'

■ **Tip:** Constraints keep your data clean and prevent invalid entries. Always add NOT NULL to columns that must have values, and UNIQUE to emails / usernames.

5 · SQL Syntax Overview

SELECT ... FROM

Read / retrieve data

```
SELECT first_name, email FROM customers;
```

WHERE

Filter rows by condition

```
SELECT * FROM orders WHERE total > 100;
```

ORDER BY

Sort results (ASC / DESC)

```
SELECT * FROM products ORDER BY price DESC;
```

JOIN

Combine rows from two tables

```
SELECT o.id, c.first_name  
FROM orders o  
JOIN customers c ON o.customer_id = c.id;
```

INSERT INTO

Add a new row

```
INSERT INTO customers (first_name, email)  
VALUES ('Alice', 'alice@email.com');
```

UPDATE ... SET

Modify existing rows

```
UPDATE customers SET city = 'Pretoria' WHERE id = 1;
```

DELETE FROM

Remove rows

```
DELETE FROM orders WHERE status = 'cancelled';
```

CREATE TABLE

Make a new table

```
CREATE TABLE products (  
  id INT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  price DECIMAL(10,2)  
);
```

ALTER TABLE

Change a table's structure

```
ALTER TABLE customers ADD COLUMN phone VARCHAR(20);
```

DROP TABLE

Delete a table permanently

```
DROP TABLE old_logs;
```

CREATE DATABASE

Create a new database

```
CREATE DATABASE shop_db;
```

DROP DATABASE

Delete an entire database

```
DROP DATABASE old_db;
```

6 · Filtering with WHERE

Operator	Meaning	Example
=	Equals	WHERE age = 25
<> or !=	Not equal	WHERE status != 'closed'
> / <	Greater / Less than	WHERE price > 50
>= / <=	Greater or Equal / Less or Equal	WHERE score >= 80
BETWEEN	Within a range (inclusive)	WHERE age BETWEEN 18 AND 65
LIKE	Pattern match (% = wildcard)	WHERE name LIKE 'AI%'
IN	Matches any value in a list	WHERE city IN ('JHB', 'CPT')
IS NULL	Column has no value	WHERE email IS NULL
AND / OR	Combine conditions	WHERE age > 18 AND city = 'JHB'
NOT	Negates a condition	WHERE NOT status = 'active'

7 · Sorting with ORDER BY

ORDER BY sorts your results. Default is **ASC** (A→Z, low→high).
Use **DESC** to reverse. You can sort by multiple columns.

```
SELECT first_name, city, age
FROM   customers
ORDER BY city ASC, age DESC;  -- sort by city first, then age inside each city
```

8 · Aliases (AS)

AS gives a column or table a temporary nickname in your results.
Makes output more readable and is required when using calculated columns.

```
-- Column alias
SELECT first_name AS 'First Name', last_name AS 'Surname'
FROM   customers;

-- Table alias (very useful with JOINS)
SELECT c.first_name, o.total
FROM   customers AS c
JOIN   orders     AS o ON c.id = o.customer_id;
```

9 · Built-in Aggregate Functions

Function	What it returns	Example
COUNT(col)	Number of non-NULL rows	SELECT COUNT(id) FROM orders;
COUNT(*)	Total rows (including NULLs)	SELECT COUNT(*) FROM customers;

SUM(col)	Total of all values	SELECT SUM(total) FROM orders;
AVG(col)	Average (mean) of all values	SELECT AVG(price) FROM products;
MAX(col)	Highest value	SELECT MAX(salary) FROM employees;
MIN(col)	Lowest value	SELECT MIN(price) FROM products;

Using GROUP BY with aggregate functions:

```
-- Total revenue per city
SELECT  c.city,
        COUNT(o.id)   AS total_orders,
        SUM(o.total)  AS revenue,
        AVG(o.total)  AS avg_order
FROM    customers c
JOIN    orders      o ON c.id = o.customer_id
GROUP BY c.city
ORDER BY revenue DESC;
```

■

Quick-Reference Card

Topic	Key Points
Database	Organised data store; better than spreadsheets for scale & integrity
Table	Rows = records, Columns = fields, Primary Key = unique row ID
Relationships	1:1, 1:N, M:N (use a junction table for M:N)
Constraints	PRIMARY KEY, NOT NULL, UNIQUE, CHECK, FOREIGN KEY, DEFAULT
Index	Speeds up queries; created automatically on PK
SELECT	Read data — always pair with FROM
WHERE	Filter rows using =, !=, LIKE, BETWEEN, IN, IS NULL, AND/OR
ORDER BY	Sort results; ASC (default) or DESC
AS	Rename a column or table in output
Aggregates	COUNT, SUM, AVG, MAX, MIN — use with GROUP BY
JOIN	Combine tables on a matching column (Foreign Key → Primary Key)
INSERT / UPDATE / DELETE	Create, modify, remove rows
CREATE / ALTER / DROP	Manage table and database structure

■

Golden Rule: Always use a WHERE clause with UPDATE and DELETE, or you'll change/delete every row in the table!